

DEV2 – JAVL – Laboratoire Java**TD 06 – Le débogage****Résumé**

Lors du laboratoire précédent, nous avons vu comment détecter les erreurs de notre code grâce à JUNIT. Quand une erreur est détectée, c'est parfois évident de trouver comment arranger le programme, mais pas toujours. Dans les situations non triviales, un débogueur va nous aider à comprendre exactement ce qui se passe.

Apprendre à utiliser un débogueur est une compétence essentielle à acquérir par tout bon programmeur.

Table des matières

1	Introduction	2
2	Le débogueur	2
3	Exercices	5

1 Introduction

Imaginons que nous avons lancé nos tests et qu'il y en a un qui ne passe pas. Il va falloir corriger le code. Comment ?

1. La première étape est de bien lire tout ce que le test nous dit : Qu'a-t-on cherché à tester ? Quelle a été la mauvaise réponse fournie ? Parfois, ça suffit à comprendre notre erreur et on peut directement la corriger.
2. Si ce n'est pas le cas, il va falloir lire attentivement la méthode erronée et essayer de comprendre ce qui se passe mal. Il ne faut pas y passer trop de temps. Généralement, si on ne trouve pas après 2-3 minutes, c'est qu'une partie du code ne fait pas ce qu'on pense qu'elle fait et il vaut mieux changer de stratégie.
3. Certains sont alors tentés d'ajouter des affichages pour vérifier :
 - ▷ si la méthode passe bien par où elle est censée passer :

```
System.out.println("On passe par ici");
```
 - ▷ si la variable a bien la valeur qu'on croit :

```
System.out.println("n vaut "+n);
```

Cette approche n'est pas recommandée. Elle oblige à modifier le code pour ensuite revenir en arrière quand on a trouvé. De plus, elle prend souvent plus de temps que l'utilisation d'un débogueur que nous allons maintenant vous présenter.

2 Le débogueur

Tout bon IDE propose un débogueur pour aider à comprendre et corriger un programme et INTELLIJ ne déroge pas à la règle.

Débogueur

Un débogueur est un outil de développement très spécialisé qui s'attache à votre application en cours d'exécution et vous permet d'inspecter votre code.

Grâce au débogueur, nous allons pouvoir exécuter le programme pas à pas et voir évoluer le contenu de chaque variable.

Tutoriel 1

Initiation au débogueur

Voici un petit tutoriel pour apprendre les bases du débogueur.

- ✍ Créez un nouveau projet et placez-y le code (inintéressant) suivant

```
1 public class Tuto1 {
2     public static void main(String[] args) {
3         int n1 = 10;
4         int n2 = 21;
5         int res;
6         if (n1%2 == n2%2) {
7             res = n2*n1;
8         } else {
9             res = n2-n1+1;
10        }
11        System.out.println(res);
12    }
13 }
```

- ✍ Exécutez-le une fois pour vérifier que tout fonctionne. Il devrait afficher 12.
- ✍ Lancez cette fois le programme en mode **debug** via le bouton .

Euh ! Le résultat est toujours le même ! ? Oui, car nous n'avons pas mis de **point d'arrêt**. Un point d'arrêt est posé sur une ligne de code et indique au débogueur de faire une pause à cet endroit.

- ✍ Placez un point d'arrêt (rond rouge) sur l'instruction 3 en cliquant dans la zone grisée à gauche de l'instruction.

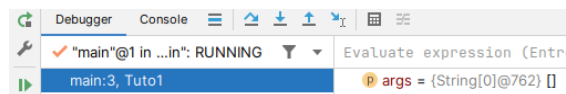
```
3  int n1 = 10;
```

- ✍ Relancez le programme en mode debug.

Cette fois le programme fait une pause à l'endroit indiqué. D'une part, il surligne dans le code la **prochaine** instruction à exécuter.

```
2  public static void main(String[] args) { args: []  
3  int n1 = 10;  
4  int n2 = 21;
```

D'autre part, il affiche toutes les variables et leurs valeurs (uniquement **args** pour le moment).



À nous de le faire avancer via les boutons dont les plus utiles sont :

- ▷ : exécute complètement l'instruction courante.
- ▷ : lorsque l'instruction courante contient un appel de méthode, on demande d'entrer dans la méthode appelée et de faire une pause au début de celle-ci (au contraire du bouton précédent qui va l'exécuter complètement).
- ▷ : continue jusqu'à l'instruction où a été positionné le curseur.
- ▷ : reprend l'exécution du programme jusqu'au prochain point d'arrêt.
- ▷ : arrête le débogueur.

- ✍ Cliquez sur le bouton et constatez
 - ▷ qu'il a avancé d'une instruction ;
 - ▷ que la variable **n1** est apparue dans la fenêtre en bas avec sa valeur actuelle.
- ✍ Continuez ainsi jusqu'à ce qu'il arrive au **if** et remarquez qu'il ne s'arrête pas sur une simple déclaration.

Parfois, on ne comprend pas trop le résultat d'un long calcul. Le débogueur nous aide en affichant le résultat d'une partie de l'expression lorsqu'on arrête la souris dessus.

- ✍ Placez la souris à différents endroits de la condition du **if** afin d'afficher : les valeurs des variables, les résultats des *modulos* et le résultat de l'égalité.

On peut aussi directement entrer une expression dans le champ prévu à cet effet.

```
n1%2  
result = 0  
args = {String[0]@760} []  
n1 = 10  
n2 = 21
```

- ✍ Testez en demandant à évaluer **n1/2**.
- ✍ Placez maintenant le curseur sur la dernière instruction (l'affichage) et demandez au débogueur de continuer jusque là.

✎ Arrêtez le débogueur.

3 Exercices

Voici quelques exercices pour mieux prendre en main le débogueur et vous inciter ainsi à l'utiliser à chaque fois que vous devrez déboguer un programme.

Exercice 1

Fibonacci

Voici un petit programme qui calcule le n^e nombre de Fibonacci.

```
1 public class Ex1 {
2
3     /**
4      * Display the nth Fibonacci number.
5      * @param n n>1
6      */
7     public static void fibonacci(int n) {
8         int n1 = 1;
9         int n2 = 1;
10        for(int i=2; i<=n; i++) {
11            int n3 = n1+n2;
12            n1 = n2;
13            n2 = n3;
14        }
15        System.out.println(n2);
16    }
17
18    public static void main(String[] args) {
19        fibonacci(20);
20    }
21 }
```

Utilisez le débogueur pour connaître la valeur des variables **n1** et **n2** après chaque passage dans la boucle.

Exercice 2

Tableau palindrome

Un tableau est considéré comme un palindrome si les valeurs sont les mêmes qu'on lise le tableau de gauche à droite ou de droite à gauche.

Voici une méthode qui a été écrite pour tester si un tableau est un palindrome.

```
1 public static boolean isPalindrome(int[] tab) {
2     for(int i=0; i<tab.length; i++) {
3         if (tab[i] != tab[tab.length-1-i]) {
4             return false;
5         } else {
6             return true;
7         }
8     }
9     return true;
10 }
```

Et voici le plan de tests qui a été imaginé avec le résultat lorsqu'on teste notre méthode.

n° du test	entrées	résultat attendu	note	réussi ?
1	{1,2,3,2,1}	true	palindrome	
2	{1,2,3,4}	false	pas palindrome	
3	{1,2,3,4,1}	false	début seulement	

Vous pouvez constater qu'il ne passe pas dans le dernier cas.

On vous demande

1. d'écrire les tests unitaires décrits ci-dessus ;
2. de vérifier que le résultat est bien celui annoncé ;

3. d'exécuter les tests en mode debug pour suivre à la trace l'évolution de la méthode dans le cas erroné ; (mettez un point d'arrêt dans la méthode de test erronée)
4. de corriger le programme ;
5. de vérifier que tous les tests passent à présent ;
6. de lancer le 1^{er} test en mode debug afin de voir si, bien que correct, il n'y aurait pas moyen de l'améliorer (performance).